

Практична робота 3

Тема: REST API з Express.js та інтеграційне тестування

Мета: набуття практичних навичок розробки RESTful API на Express.js з TypeScript та написання інтеграційних тестів з використанням Supertest

Хід роботи:

Варіант 5

Завдання 1.

Створіть новий Node.js-проект з усіма необхідними залежностями

Було ініціалізовано проект та встановлено відповідні залежності.

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				1
Змн.	Арк.	№ докум.	Підпис	Дата		

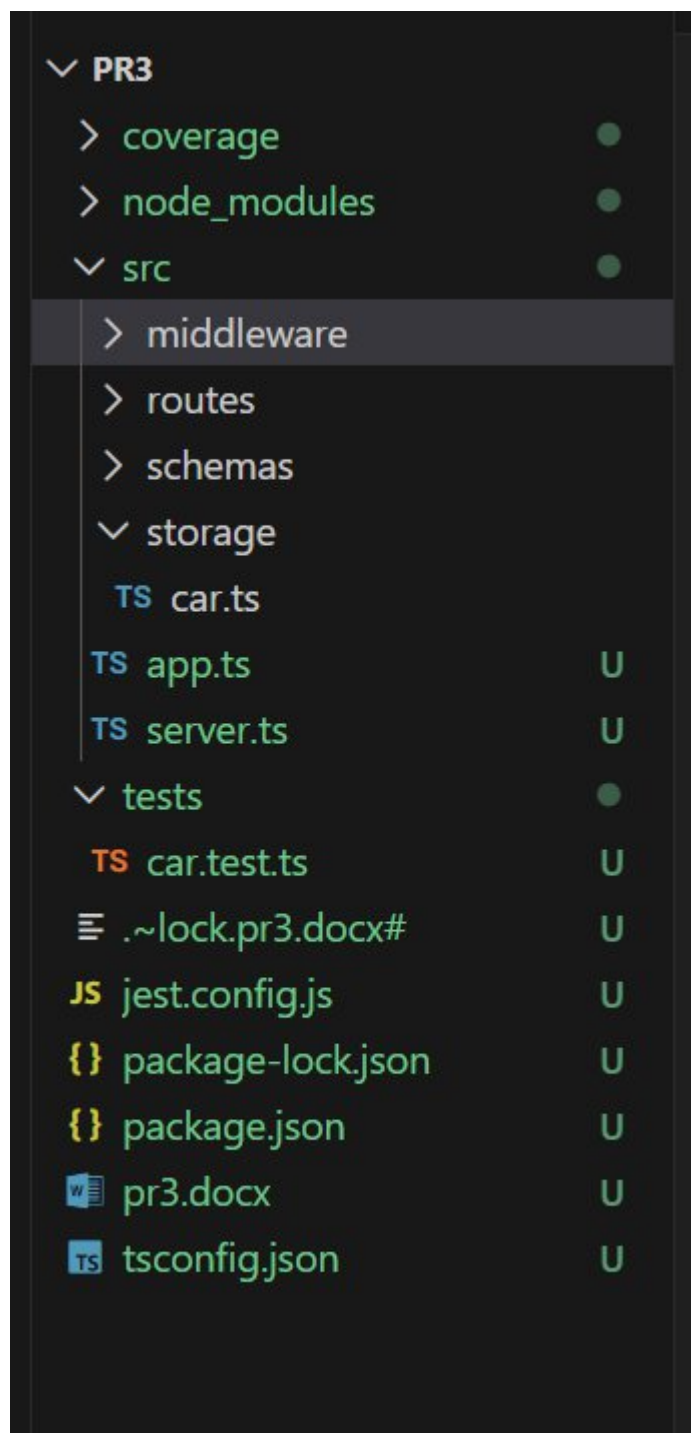


Рис. 3.1 – Структура проекту

Завдання 2

Створіть схему для роботи з обраною сутністю. Оберіть сутність із домену, який вам знайомий — книга, рецепт, фільм, гравець, планета тощо. Чим краще ви розумієте предметну область, тим легше вийде зробити схему. Не треба повторювати приклад з лекції :)

Вимоги:

Схема створення (createSchema) повинна містити:

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

- обов'язкове текстове поле з назвою, що відповідає вашій сутності (title для книги, name для персонажа, designation для планети тощо) — 1–100 символів;
- опціональне поле опису — до 500 символів; ► мінімум 2 додаткових поля, що відображають реальні атрибути вашої сутності. Поле має відповідати на питання: «що корисно знати про цей об'єкт?» Наприклад, для книги це може бути рік видання (число зі зрозумілими обмеженнями) та жанр (список заготовлених значень); для рецепту — час приготування в хвилинах та рівень складності. Поля типу isActive чи count без конкретного змісту не зараховуються.

Схема оновлення (updateSchema) — всі поля опціональні.

Тип збереженої сутності (Entity) — виводиться з createSchema (z.infer) і розширюється серверними полями: id, createdAt, updatedAt.

```
src > schemas > TS car.schema.ts > ...
1  import { z } from 'zod';
2
3  const currentYear = new Date().getFullYear();
4  ⚡
5  export const createCarSchema = z.object({
6    name: z.string().min(1).max(100),
7    brand: z.string().min(1).max(40),
8    model: z.string().min(1).max(40).transform(s => s.trim()),
9    year: z.number().int().min(1920).max(currentYear),
10   transmission: z.enum(['manual', 'automatic']),
11   price: z.number().positive(),
12   description: z.string().max(500).optional(),
13 });
14
15 export const updateCarSchema = createCarSchema.partial();
16
17 export type CreateCarDTO = z.infer<typeof createCarSchema>;
18 export type UpdateCarDTO = z.infer<typeof updateCarSchema>;
19
20 export type Car = CreateCarDTO & {
21   id: string;
22   createdAt: Date;
23   updatedAt: Date;
24 };
```

Рис. 3.2 – Створення схеми сутності

Завдання 3

Реалізуйте CRUD-модуль на основі Map

Вимоги:

- Реалізуйте модуль на основі Map. Назви функцій та їх сигнатури — на ваш розсуд.
- Модуль повинен підтримувати: отримання всіх записів, отримання за id, створення, оновлення (з оновленням updatedAt), видалення.
- Передбачте можливість скидання стану сховища — вона знадобиться для ізоляції тестів

```
src > storage > TS car.ts > CarFilters
1 import { Car, CreateCarDTO, UpdateCarDTO } from '../schemas/car.schema';
2 import { randomUUID } from 'crypto';
3
4 const cars = new Map<string, Car>();
5
6 type CarFilters = {
7   brand?: string;
8   transmission?: 'manual' | 'automatic';
9 };
10
11
12 export function getAllCars(filters?: CarFilters): Car[] {
13   let result = Array.from(cars.values());
14
15   if (filters?.brand) {
16     result = result.filter((car) => car.brand.toLowerCase() === filters.brand!.toLowerCase());
17   }
18
19   if (filters?.transmission) {
20     result = result.filter((car) => car.transmission === filters.transmission);
21   }
22
23   return result;
24 }
25
26 export function getModernCars(): Car[] {
27   return Array.from(cars.values()).filter((car) => car.year >= 2015);
28 }
```

Рис. 3.3 – Створення CRUD на основі Map

Завдання 4

Реалізуйте Express Router з повним набором CRUD-маршрутів, validate middleware та errorHandler.

Вимоги:

- Реалізуйте validate(schema) middleware — парсить тіло запиту через схему і викидає помилку, якщо дані невалідні.
- Реалізуйте errorHandler — розрізняйте помилки валідації (ZodError, статус 400) та інші помилки (статус 500). Формат відповіді при помилці — на ваш розсуд, але він має бути інформативним.

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

- Підтримайте операції: отримання всіх записів, отримання за id, створення, оновлення, видалення.
- Обирайте статус-коди відповідно до HTTP-семантики: 200, 201, 204, 400, 404 тощо.
- При створенні генеруйте id через `crypto.randomUUID()`, виставляйте дату `createdAt/updatedAt`

```
src > middleware > TS validate.ts > validate
1  import { ZodObject } from 'zod';
2  import { Request, Response, NextFunction } from 'express';
3
4  export function validate(schema: ZodObject) {
5    return (req: Request, res: Response, next: NextFunction): void => {
6      try{
7        req.body = schema.parse(req.body);
8        next();
9      } catch(error) {
10       next(error);
11     }
12   };
13 }
```

```
src > middleware > TS errorHandler.ts > errorHandler
1  import { ZodError } from 'zod';
2  import { Request, Response, NextFunction } from 'express';
3
4  export function errorHandler(
5    err: Error,
6    req: Request,
7    res: Response,
8    next: NextFunction
9  ) {
10   if (err instanceof ZodError) {
11     return res.status(400).json({
12       errors: err.issues.map(i => ({
13         field: i.path.join('.'),
14         message: i.message,
15       })),
16     });
17   }
18 }
```

Рис. 3.4 – Реалізація middlewares

Завдання 5

		Ліщинський В. В.			ДУ «Житомирська політехніка».24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

Розділіть конфігурацію Express та запуск сервера.

Вимоги:

- app.ts — створює і налаштовує Express, не викликає listen(), експортує app.
- server.ts — імпортує app, викликає listen().
- Дотримуйтесь правильного порядку підключення middleware.

```
src > TS app.ts > ...
1  import express from 'express';
2  import carRouter from './routes/car';
3  import { errorHandler } from './middleware/errorHandler';
4
5  const app = express();
6  app.use(express.json());
7
8  app.use('/api/cars', carRouter);
9  app.use(errorHandler);
10
11 export default app;
```

```
src > TS server.ts > ...
1  import app from './app';
2
3  const PORT = process.env.PORT || 3000;
4
5  app.listen(PORT, () => {
6    | console.log(`Server running on port ${PORT}`);
7  });|
```

Рис. 3.5-3.6 – app.ts та server.ts

Завдання 6

Напишіть інтеграційні тести для вашого API.

Вимоги:

- Використовуйте beforeEach для очистки пам'яті та ізоляції тестів.
- Покрийте тестами всі маршрути: успішні сценарії та граничні випадки (не знайдено, невалідні дані тощо).
- Перевіряйте статус-коди, структуру відповідей і заголовки там, де це доречно.
- Загальна кількість тестів — не менше 12.

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				6
Змн.	Арк.	№ докум.	Підпис	Дата		


```

Cars API
GET /api/cars
  ✓ should return empty array when there are no cars (12 ms)
  ✓ should return all cars (21 ms)
Filtering and custom route
  ✓ should filter cars by brand (9 ms)
  ✓ should filter cars by transmission (9 ms)
  ✓ should filter cars by brand and transmission together (9 ms)
  ✓ should return modern cars only (8 ms)
POST /api/cars
  ✓ should create a car (2 ms)
  ✓ should return 400 for invalid car data (2 ms)
  ✓ should return 400 when required fields are missing (2 ms)
GET /api/cars/:id
  ✓ should return a car by id (3 ms)
  ✓ should return 404 if car is not found (2 ms)
PATCH /api/cars/:id
  ✓ should update a car (4 ms)
  ✓ should return 404 when updating non-existing car (2 ms)
  ✓ should return 400 for invalid update data (3 ms)
  ✓ should allow partial update (3 ms)
DELETE /api/cars/:id
  ✓ should delete a car (4 ms)
  ✓ should return 404 when deleting non-existing car (1 ms)

Test Suites: 1 passed, 1 total
Tests:       17 passed, 17 total
Snapshots:   0 total
Time:        0.623 s, estimated 2 s

```

Рис. 3.7 – Створені тести

Завдання 7

Розширте додаток підтримкою фільтрації та напишіть тести. Фільтрацію слід реалізувати на рівні сховища, а не в обробнику маршруту: функція отримання всіх записів приймає опціональні параметри та повертає вже відфільтрований результат. Це правильний поділ відповідальності — у наступних темах, коли сховище замінимо на базу даних, фільтрація природно перетвориться на умови запиту, не змінюючи код маршрутів.

Вимоги:

- Реалізуйте щонайменше 2 query-параметри для фільтрації/пошуку, що мають сенс для вашої предметної області. Назви параметрів обирайте самостійно.
- Параметри можна комбінувати.
- Напишіть тести для кожного параметра та їх комбінацій.

- Додайте один додатковий маршрут з власним шляхом, специфічний для вашої предметної області. Наприклад: GET /planets/habitable, GET /books/bestsellers, GET /recipes/quick. Покрийте його тестами

```
type CarFilters = {
  brand?: string;
  transmission?: 'manual' | 'automatic';
};

export function getAllCars(filters?: CarFilters): Car[] {
  let result = Array.from(cars.values());

  if (filters?.brand) {
    result = result.filter((car) => car.brand.toLowerCase() === filters.brand!.toLowerCase());
  }

  if (filters?.transmission) {
    result = result.filter((car) => car.transmission === filters.transmission);
  }

  return result;
}

export function getModernCars(): Car[] {
  return Array.from(cars.values()).filter((car) => car.year >= 2015);
}
```

Рис. 3.8 – Додані фільтри

Загальні вимоги

1. Весь код написано з використанням TypeScript.
2. Типи сутності максимально виводяться через z.infer.
3. Для валідації використовується Zod — ніякої ручної валідації через if.
4. Правильні HTTP-статус-коди: 200 (GET, PATCH), 201 (POST), 204 (DELETE), 400 (невалідні дані), 404 (ресурс не знайдено) тощо.
5. Обробка помилок через middleware.
6. Завантажте код на GitLab та надайте доступ викладачу.

Висновок

На даній лабораторній роботі ми набули практичних навичок розробки RESTful API на Express.js з TypeScript та написання інтеграційних тестів з використанням Supertest.

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				8
Змн.	Арк.	№ докум.	Підпис	Дата		